Graphs and Graph Searches

DSA27 — Lecture 14 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Week 14 · Fall 2026

Where We Are

The last structure

Every structure so far was a graph with rules:

- a **linked list**: each node points to at most one next node;
- a **tree**: one root, one parent per node, no cycles;
- a **graph**: nodes and edges, **no rules at all**.

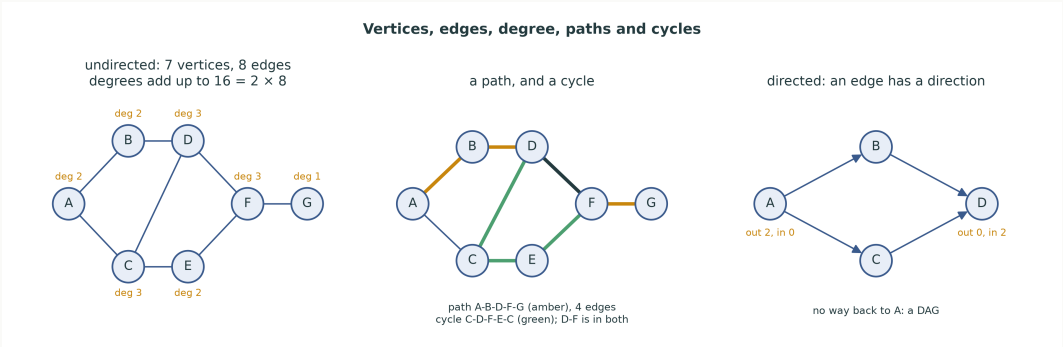
Today: store one, then **search** it — breadth first and depth first.

Today

1. The words: vertex, edge, degree, path, cycle, component, DAG
2. Two representations: adjacency list and adjacency matrix — measured
3. Breadth-first search, with a queue — and shortest paths
4. Depth-first search, recursive and with your own stack
5. Components, cycles, topological sort
6. A maze, and your own degree's prerequisites

The Vocabulary

Vertices and edges

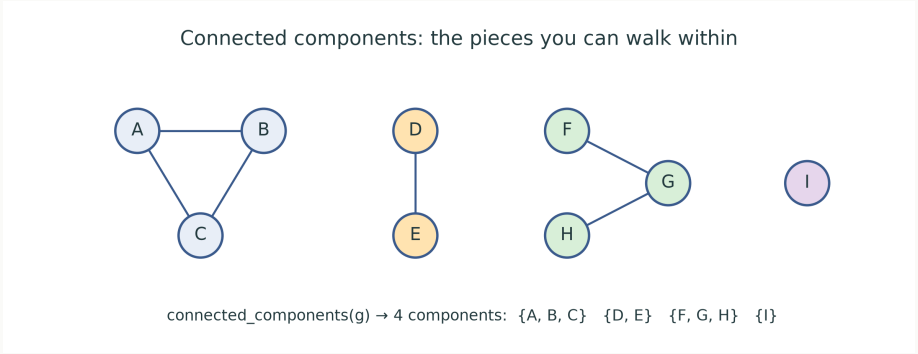


- **Vertex** (node) — **الرأس**. **Edge** — **الحافة**. V vertices, E edges.
- **Undirected**: an edge goes both ways. **Directed**: it has a direction.
- **Degree** — **الدرجة**: the number of edges at a vertex.

Paths, cycles and components

- **Path** — المسار: a sequence of vertices, each joined to the next by an edge. Its **length** is its number of edges.
- **Cycle** — الدور: a path that returns to where it started.
- **Connected** — every vertex can reach every other. A **connected component**, المركّبة المتصل, is a largest connected piece.
- **DAG**, مخطط موجَّه لا دوري: a directed graph with **no** cycle.

Connected components



One search from a vertex visits **exactly its component**. To find all components: start a new search from every vertex not yet seen.

Graphs are everywhere

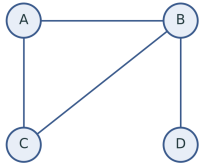
Problem	Vertices	Edges	Kind
Maze, game map	open cells	neighbouring cells	undirected
Road map	junctions	roads	(un)directed
Web	pages	links	directed
Your degree	courses	“is a prerequisite of”	DAG
Build, installs	files, packages	“is needed by”	DAG

Two Representations

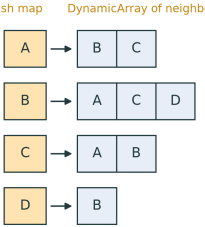
Adjacency list and adjacency matrix

One graph, two representations

the graph



adjacency list: V + 2E = 12 cells



adjacency matrix: V² = 16 cells

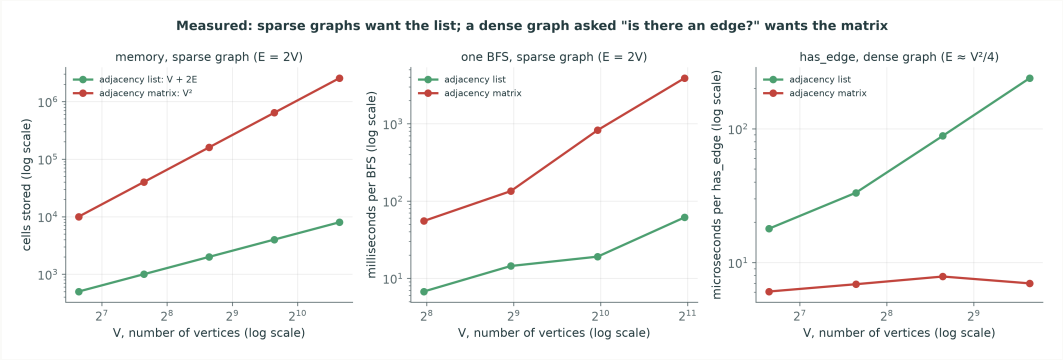
	A	B	C	D
A	0	1	1	0
B	1	0	1	1
C	1	1	0	0
D	0	1	0	0

- **List**: for each vertex, the vertices it reaches.
- **Matrix**: a V × V grid; cell [i][j] is 1 when there is an edge i → j.

The trade-off

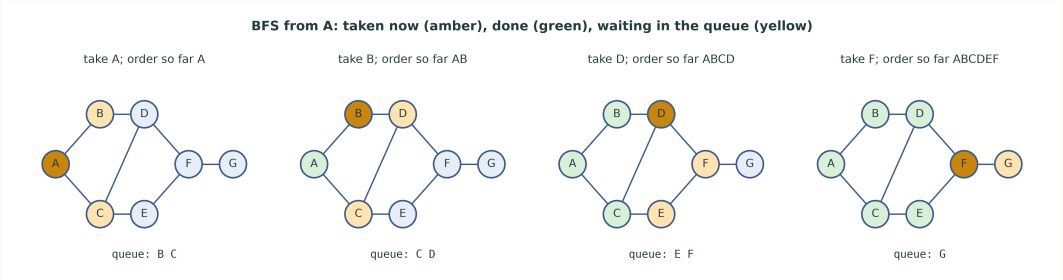
	Adjacency list	Adjacency matrix
Memory	$O(V + E)$	$O(V^2)$
`has_edge(u, v)`	$O(\deg u)$	$O(1)$
`neighbours(u)`	$O(\deg u)$	$O(V)$ — scan the row
`add_node`	$O(1)$	$O(V)$ — a new column
One BFS or DFS	$O(V + E)$	$O(V^2)$

Measured


Measured: sparse graphs want the list; a dense graph asked "is there an edge?" wants the matrix
memory, sparse graph (E = 2V)
adjacency list: V + 2E
adjacency matrix: V²
cells stored (log scale)
V, number of vertices (log scale)
one BFS, sparse graph (E = 2V)
adjacency list
adjacency matrix
milliseconds per BFS (log scale)
V, number of vertices (log scale)
has_edge, dense graph (E ≈ V²/4)
adjacency list
adjacency matrix
microseconds per has_edge (log scale)
V, number of vertices (log scale)

Breadth-First Search

Rings round the start



Visit the start, then **everything 1 edge away**, then everything 2 away, …

A **queue** keeps the frontier in order: first discovered, first expanded.

The algorithm

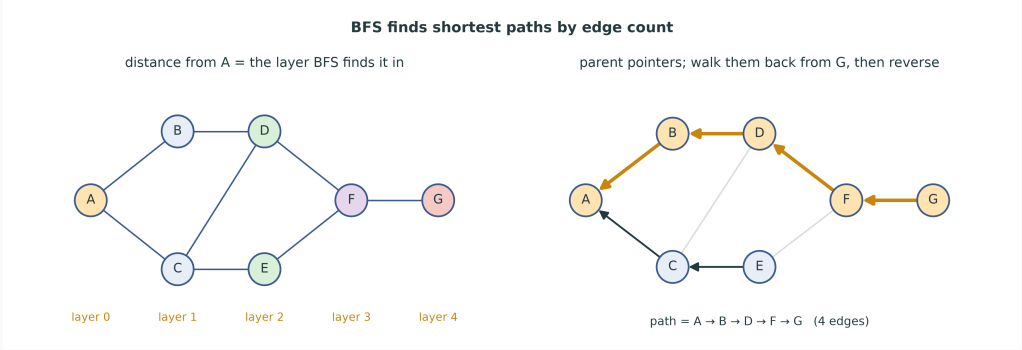
```
bfs(graph, start)
    order   = empty list
    visited = empty hash map;  visited[start] = True
    queue   = empty CircularQueue;  enqueue start
    while the queue is not empty:
        node = dequeue
        append node to order
        for each neighbour of node, in graph.neighbours(node) order:
            if neighbour is not in visited:
                visited[neighbour] = True      # mark when queued
                enqueue neighbour
    return order
```

Each vertex is enqueued once and dequeued once: $O(V + E)$ on a list.

Trace: `bfs(G, "A")`

Take	Discover (enqueue)	Queue afterwards	Order so far
A	B, C	B C	A
B	D (A is visited)	C D	A B
C	E (A, D are visited)	D E	A B C
D	F	E F	A B C D
E	— (C, F are visited)	F	A B C D E
F	G	G	A B C D E F
G	—	empty	A B C D E F G

Shortest paths, by edge count

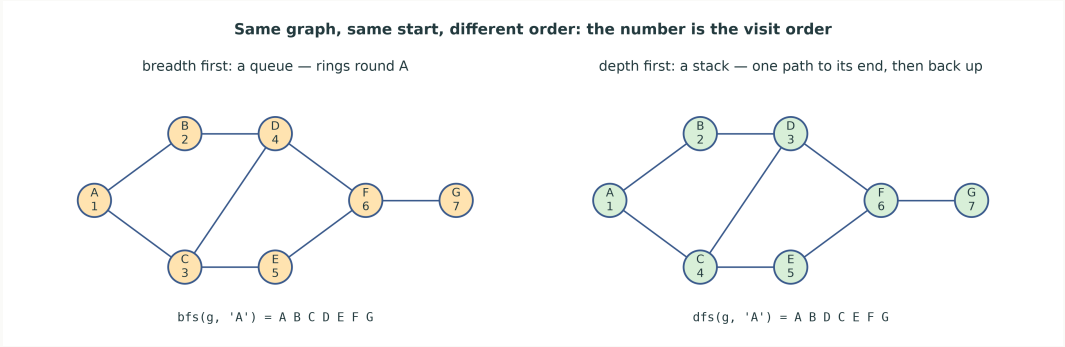


The first time BFS reaches a vertex, it has come by a **shortest** path.

Remember **who discovered whom** (a parent map), then walk back from the goal.

Depth-First Search

One path to its end, then back up



DFS goes **deep** first: follow a neighbour, then its neighbour, … until stuck; then **back up** to the last vertex with an unvisited neighbour.

Recursive DFS: the call stack does the work

```
dfs(graph, start)
    order = empty list;  visited = empty hash map
    visit(start)
    return order

visit(node)
    visited[node] = True
    append node to order
    for each neighbour of node, in graph.neighbours(node) order:
        if neighbour is not in visited:
            visit(neighbour)
```

$O(V + E)$ on a list — and $O(V)$ **stack** in the worst case.

DFS with your own stack

```
dfs_iterative(graph, start)
    order = empty list;  visited = empty hash map
    stack = empty Stack;  push start
    while the stack is not empty:
        node = pop
        if node is in visited:  continue    # a second copy
        visited[node] = True                # mark when taken
        append node to order
        for each neighbour, in REVERSE order:
            if it is not in visited:  push it
    return order
```

Same order as `dfs`, and no recursion limit.

Why the two orders can differ

"Mark when pushed", BFS-style, with a stack:

Variant	Order from A
recursive `dfs`	A B D C E F G
stack, mark on pop, push reversed	A B D C E F G
stack, mark on **push**, push reversed	A B D **F E G C**
stack, mark on push, push in order	A C E F G D B

BFS and DFS side by side

	BFS	DFS
Container	**queue** (FIFO)	**stack** (LIFO) — or the call stack
Order	rings, by distance	one path to its end, then back up
Shortest path by edges	**yes**	no
Time, adjacency list	$O(V + E)$	$O(V + E)$
Time, adjacency matrix	$O(V^2)$	$O(V^2)$
Extra memory	$O(V)$	$O(V)$ recursive; up to $O(E)$ own stack
Good for	distances, “nearest”, levels	cycles, topological order, mazes, backtracking

Using the Searches

Connected components

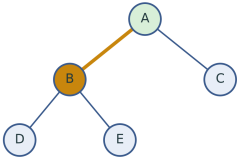
```
connected_components(graph)
    seen = empty hash map;  components = empty list
    for each node in graph.nodes():
        if node is not in seen:
            component = bfs(graph, node)
            mark every member of component as seen
            append component to components
    return components
```

$O(V + E)$ in total — each vertex is searched from once.

Cycles: undirected and directed are different

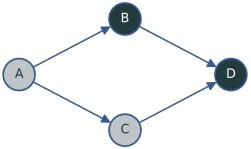
Cycle detection: white (unseen), grey (on the path), black (finished)

undirected: skip the edge you came in by



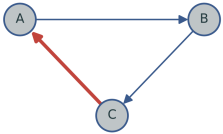
at B, neighbour A is visited — but it is B's parent: not a cycle

directed: visited is not enough



reaching D again from C: D is black (finished) — a diamond, not a cycle

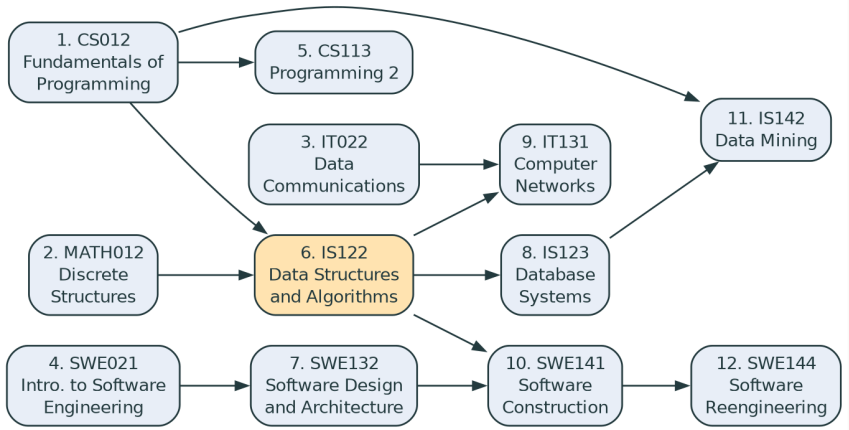
grey = on the path now: a cycle



C → A reaches a grey node: A is still on the current path — a cycle

- **Undirected:** a visited neighbour that is **not your parent** closes a cycle.
- **Directed:** a neighbour that is **grey** — still on the current path.

Topological sort



Software Engineering bylaw (2013): an arrow X → Y means X is a prerequisite of Y.
The number is the position in topological_sort(g).

An order of the vertices in which **every edge points forwards**. It exists **exactly when the graph is a DAG**.

Kahn's algorithm: take what nothing waits on

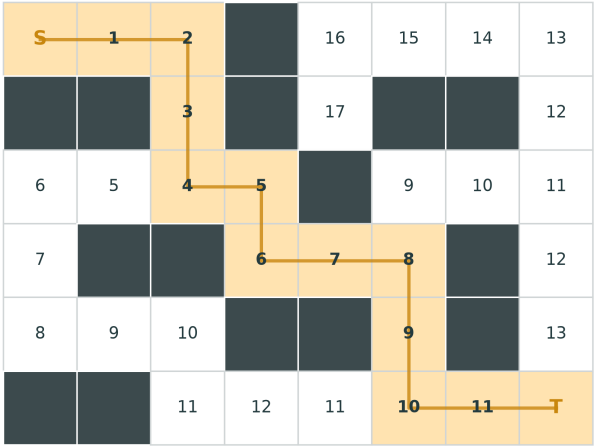
1. Count each vertex's **in-degree** — its unmet prerequisites.
2. Queue every vertex with in-degree 0.
3. Repeat: dequeue X, output it; for each X $\to$ Y, subtract 1 from Y's count; if it reaches 0, enqueue Y.
4. Output shorter than V? The rest wait on each other: **a cycle** — return `None`.

$O(V + E)$: each vertex queued once, each edge crossed once.

Two Real Graphs

A maze is a graph

A maze is a graph: each number is BFS's distance from S



33 open cells = 33 vertices, 34 edges; shortest route S → T: 12 steps

Your degree is a DAG

- **Topological sort** $\rightarrow$ a valid study plan.
- **Cycle check** $\rightarrow$ can the plan be completed at all?
- **BFS from IS122** $\rightarrow$ everything this course unlocks.
- **Layers** of Kahn's algorithm $\rightarrow$ the fewest semesters, if you could take anything that is ready.

Beyond the Bylaw

Weighted graphs: named, not examined

- **Weighted edges**: a road has a length, a link a cost.
- **Dijkstra's algorithm**: shortest paths by total weight — BFS with a **priority queue** (Week 12) instead of a queue.
- **Minimum spanning tree** (Prim, Kruskal): the cheapest set of edges that keeps a graph connected.

Neither is declared by the bylaw for this course.

This Week

Exercises: `dsa/graph.py`

Exercise	Target	The trap
`add_node`, `add_edge`	$O(1)$	adding twice keeps the edges
`neighbours`, `edges`, …	$O(\deg)$, $O(V+E)$	undirected edges reported **once**
`MatrixGraph`	see the table	a new row **and** a new column
`bfs`, `shortest_path_unweighted`	$O(V+E)$	mark on enqueue; parents
`dfs`, `dfs_iterative`	$O(V+E)$	mark on pop; push reversed
`has_cycle`, `topological_sort`	$O(V+E)$	parent against grey; `None`

```
pytest tests/test_graph.py -v
```

Homework 14 — before Lecture 15

1. **Implement** `dsa/graph.py` until all 63 tests pass.
2. **Trace** `bfs` and `dfs_iterative` from A on the test file's sample graph, with the queue or stack after every step.
3. **Measure** a BFS on `Graph` against `MatrixGraph` in `notebooks/14-graphs.ipynb`, for V = 250 to 2,000.
4. **Your degree.** Type ten prerequisites of your own programme into a directed `Graph`, and print a topological order.

Practise online

- **Week 14 quiz**: 22 multiple-choice questions, every answer explained
- **Written practice** on the same page: 5 questions. Write, then mark yourself against the marking points
- In Arabic: week 14

Summary

Seven things to keep

1. A graph is vertices and edges; lists and trees are graphs with rules.
2. **List**: $O(V + E)$ memory, neighbours in $O(\deg)$. **Matrix**: V^2 memory, `has_edge` in $O(1)$.
3. **BFS** uses a queue, visits in layers, finds shortest paths **by edge count**. Mark on enqueue.
4. **DFS** uses a stack — the call stack or yours. Own stack: mark on pop, push reversed.
5. Both are $O(V + E)$ on a list, $O(V^2)$ on a matrix.
6. Cycles: undirected — not the parent; directed — **grey**, not merely visited.
7. A topological order exists **exactly** for a DAG: Kahn, or reversed DFS finish order.

Next

Week 15 — Principles of language translation. A tokenizer, a recursive-descent parser and an evaluator: a **stack**, **recursion** and a **tree**, together — and an expression's parse tree is one more graph.